PolyGlot
A Compiler for Mixed Syntax

Review 3 Final Project Report

Course name: Compiler Design LAB

Course code: BCSE307P

Submitted by

Name	Reg. No.
Harshit Sharma Thakur	24BCE0437
Umang Nagaji Vekariya	24BAI0173
Sarthak Shirish Dhawale	24BCT0216
Aditya Mayur Khunt	24BCE2618

Faculty: Boominathan P

Vellore Institute of Technology (VIT), Vellore

Academic Year: 2026–2027

Repository: https://github.com/hst257/PolyGlot

Live site: https://hst257.github.io/PolyGlot/

Table of Contents

Sr. No.	Contents	Pg. No.
1	Abstract	3
2	Introduction	4
3	Literature Survey	5
4	Proposed System	6
5	System Architecture	7–9
6	System Design	10–12
7	Technology Stack	13
8	Code and Implementation	14–17
9	Testing and Validation	18–19
10	Results and Screenshots	20–23
11	Repository and Setup	24
12	Challenges and Solutions	25
13	Conclusion and Future Scope	26
14	References	27
15	Appendix A Selected Source Code	28–30

1. Abstract

PolyGlot is an educational compiler that accepts selected syntax from Python, C, C++, Java, JavaScript and BASIC in the same program. These forms are converted into one abstract syntax tree and checked using common rules for types, variables and functions. The project helps students follow a program from source code to execution without switching between separate language tools.

The compiler uses a handwritten lexer, recursive-descent statement parsing and Pratt expression parsing. It then performs semantic checks, generates typed three-address intermediate code, applies limited optimizations and produces stack bytecode. A custom virtual machine executes the bytecode. The browser interface displays the actual tokens, tree, symbols, intermediate code, bytecode and output.

The implemented subset supports annotated functions, recursion, conditions, loops and fixed-length one-dimensional arrays. Equivalent syntax forms share the same variables, function calls and runtime behavior. Compiled programs can also be saved as JSON .pbc files and executed without the original source.

The recorded verification at commit 9aee210 includes 298 passing Python tests and 57 passing frontend tests. For print((2 + 3) * 4), both execution modes produce 20; optimization reduces the executed instruction count from 17 to 5. This comparison measures instructions for one example, rather than execution time. The CLI, local IDE and static browser version reuse the same compiler implementation. [1]

2. Introduction

Different programming languages use different spellings for similar operations. In a compiler course, moving between those languages can make it harder to see which rules belong to syntax and which belong to program meaning. PolyGlot brings a small set of familiar forms into one compiler so their common behavior can be inspected directly.

- The main objective is to compile the documented mixed syntax through one pipeline. Tokens, source positions, syntax recognition, types and generated code remain available for inspection.
- Equivalent statements produce common AST nodes. For example, supported print forms become an Output node, and accepted function definitions use the same call and return rules.
- The IDE provides Check, Build and Run operations. Check reports compilation results, Build saves bytecode, and Run executes a valid program on the VM. The CLI uses the same underlying compiler.
- The supported values are int, float, bool, string and fixed-length arrays of primitive elements. Functions require parameter and return annotations. Conditions, while loops, range loops, three-part for loops, break and continue are included.
- The scope is deliberately limited. Complete programs from the six source languages are not accepted. Classes, imports, general member access, multidimensional arrays, slicing and first-class functions are outside the current implementation.
- The project focuses on showing how compilation works. It uses a custom VM and in-memory representations; it does not require a database or a native machine-code backend. [1]

3. Literature Survey

The following tools and references informed the project design. The useful comparisons are how they represent programs and which stages they expose.

Reference	Relevant work	Use in PolyGlot
LLVM Kaleidoscope [2]	A tutorial compiler with lexer, parser, AST and LLVM IR.	Separating compiler phases and defining their interfaces.
Pratt [3]	Top-down operator precedence for expressions.	A shared expression parser with a precedence table.
Cornell compiler notes [4]	Control-flow and live-variable analysis.	Backward liveness over IR basic blocks.
Compiler Explorer [5]	Interactive inspection of established compiler output.	Making generated code visible in the browser.
AST Explorer [6]	Parser selection and interactive AST inspection.	Displaying tree structure and source relationships.
Crafting Interpreters [7]	Language implementation and a bytecode VM.	A readable custom runtime for teaching.
Pyodide [8]	Python execution in a browser Web Worker.	Reusing the Python compiler in the static site.

- PolyGlot follows the usual separation between parsing, semantic checks, intermediate code and execution. It targets its own stack VM instead of LLVM.
- Its main project-specific feature is the combination of mixed syntax, shared semantics and visible compiler stages. The compared tools serve different purposes, so they are used as design references rather than a ranking.

4. Proposed System

PolyGlot accepts source text, a filename and an optimization option. It returns diagnostics and the stages completed for that input. Build produces a reusable artifact; Run adds program output and an instruction count. Exec loads and validates an existing artifact. [1]

Syntax family	Accepted examples	Common meaning
Python	Annotated def; print; indented suites; range and array loops.	Functions, output and control flow.
C	Typed declarations/functions; three-part for; restricted printf.	Typed bindings, functions and output.
C++	cout or std::cout chains; endl.	Output with a separator and ending.
Java	System.out.print/println; primitive aliases and array forms.	Output, types and arrays.
JavaScript	let, const, console.log; annotated function.	Bindings, output and functions.
BASIC	Uppercase LET and PRINT.	Assignment and output.

- All accepted forms share lexical scopes and fixed types. Aliases such as double, boolean and String map to float, bool and string. Integer values can widen to float where required.
- A const binding cannot be rebound through another syntax. Array elements remain mutable, and arrays keep a fixed length and one primitive element type.
- Compilation stops at the first error. Earlier stages remain available, but no artifact is returned. Runtime errors can retain partial output and available compilation data.
- Exact accepted forms are documented in docs/language.md and docs/grammar.ebnf. Shared spellings may have several syntax-family candidates; this metadata does not change their semantics.

5. System Architecture

The local IDE sends requests to a loopback HTTP server. The CLI calls the same compiler library directly. Both paths use the same bytecode validator and VM. [1, 9, 10]

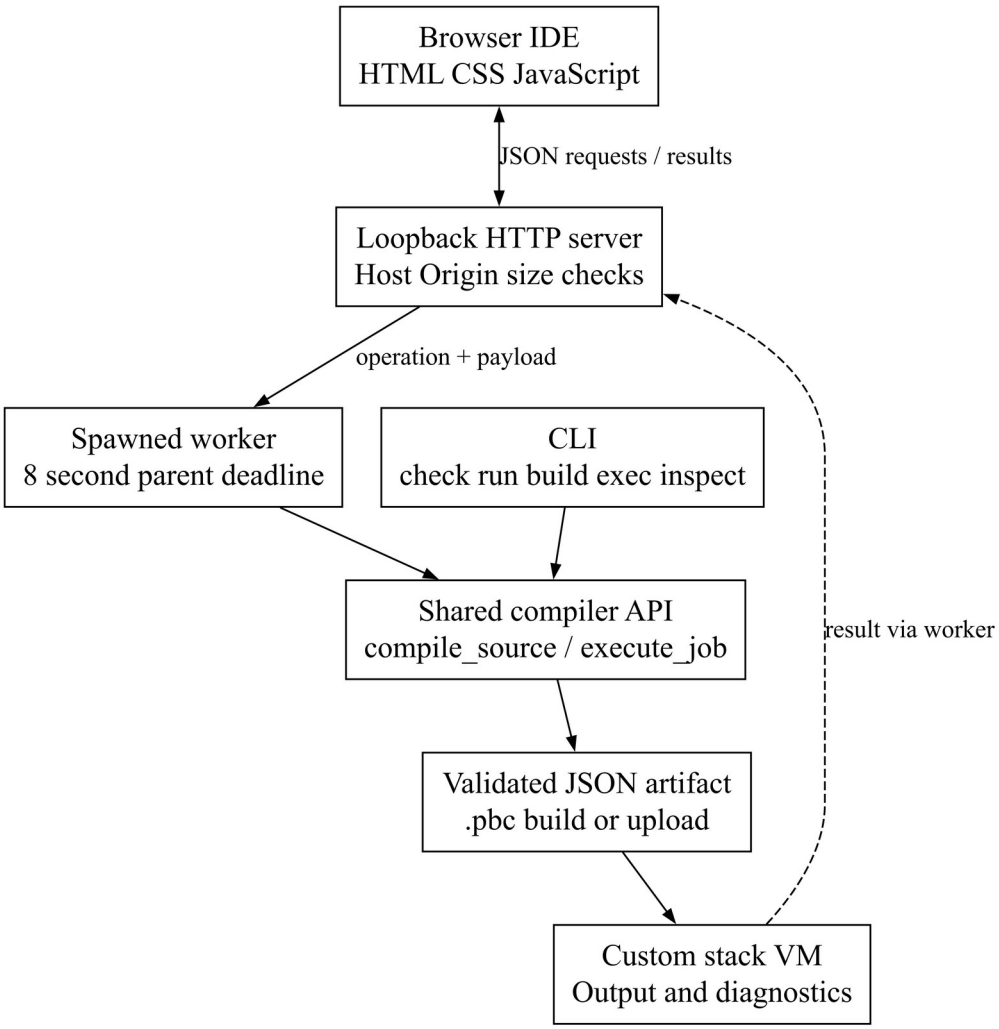


Figure 1 Local IDE and CLI architecture

- The server checks allowed routes, request size, Host and supplied Origin headers before starting a spawned worker. The worker runs execute_job and returns JSON through a pipe.
- The parent can stop a job after its default eight-second deadline. The VM has a separate five-second execution limit. Diagnostics and stage results return through the same request path.

System Architecture

The compiler passes explicit representations between stages. Semantic analysis adds types and resolved bindings to the AST before IR generation. Optimization changes the IR used by code generation while preserving the original IR for the inspector.

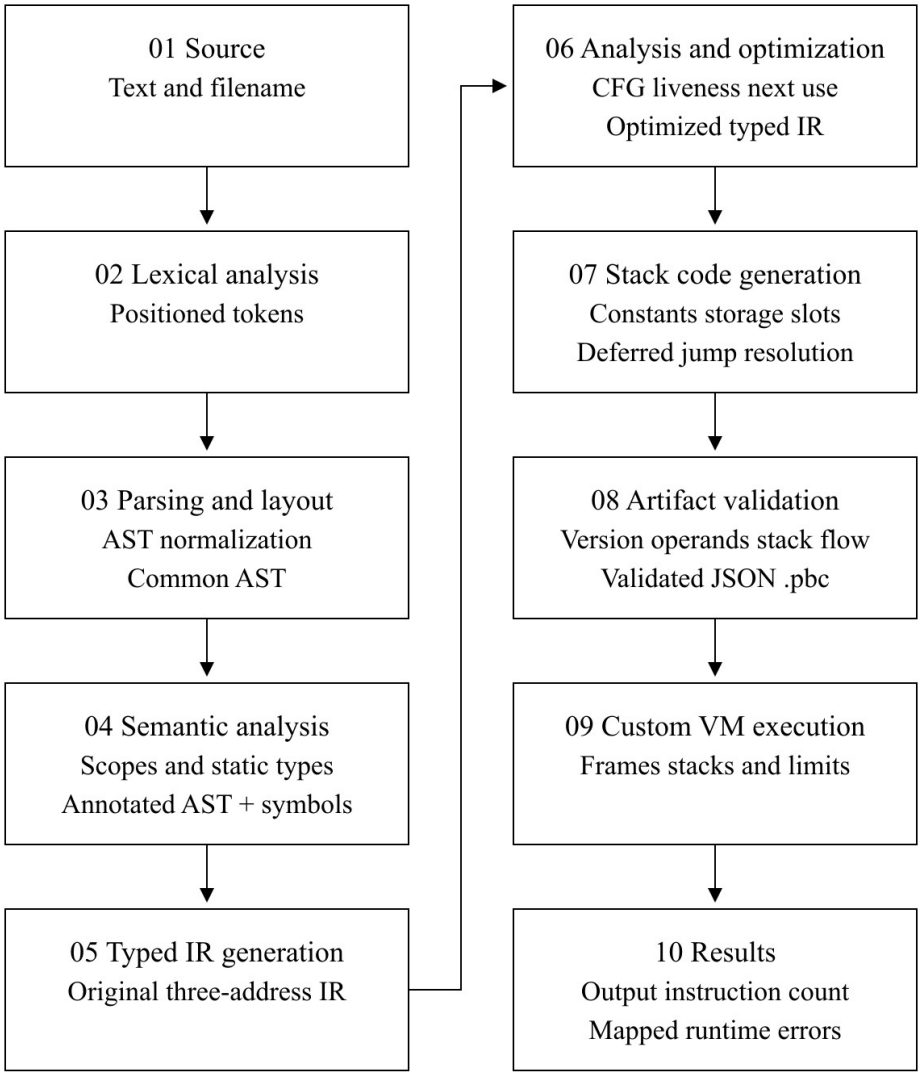


Figure 2 Compiler stages and their representations

- Tokens, AST nodes, IR operations and bytecode instructions retain source locations. This lets later diagnostics point back to the input.
- Artifact validation checks generated or uploaded bytecode before execution. The VM then performs the dynamic checks that depend on actual values.

System Architecture

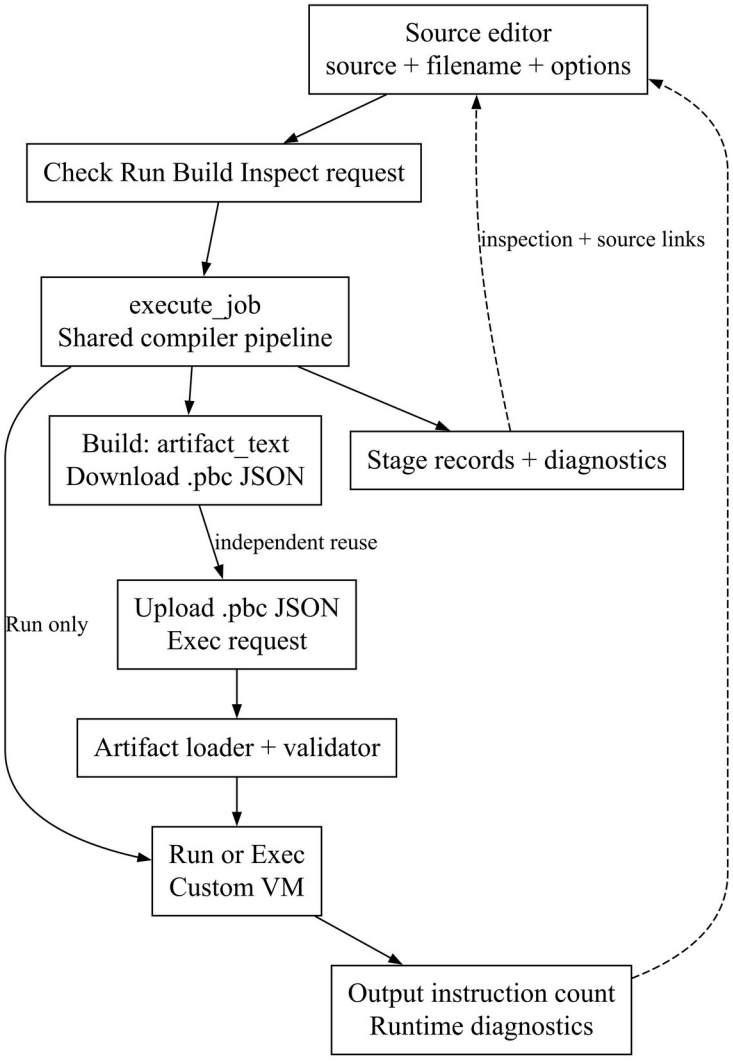


Figure 3 Source compilation and artifact reuse

- Check, Build and Inspect return compilation data. Run executes after successful compilation. An uploaded .pbc file goes through the artifact loader and validator, then directly to the VM.
- The static site packages the Python compiler and runs execute_job under Pyodide in a module Web Worker. The browser uses worker messages in place of local HTTP requests. [8]
- Editing the source or filename invalidates old inspection links. Responses from an earlier editor revision cannot attach diagnostics to newly edited code.

6 System Design

Statement parsing uses leading tokens and bounded lookahead. Distinctive forms such as LET and console.log select a specific rule; assignments and common control statements use shared rules.

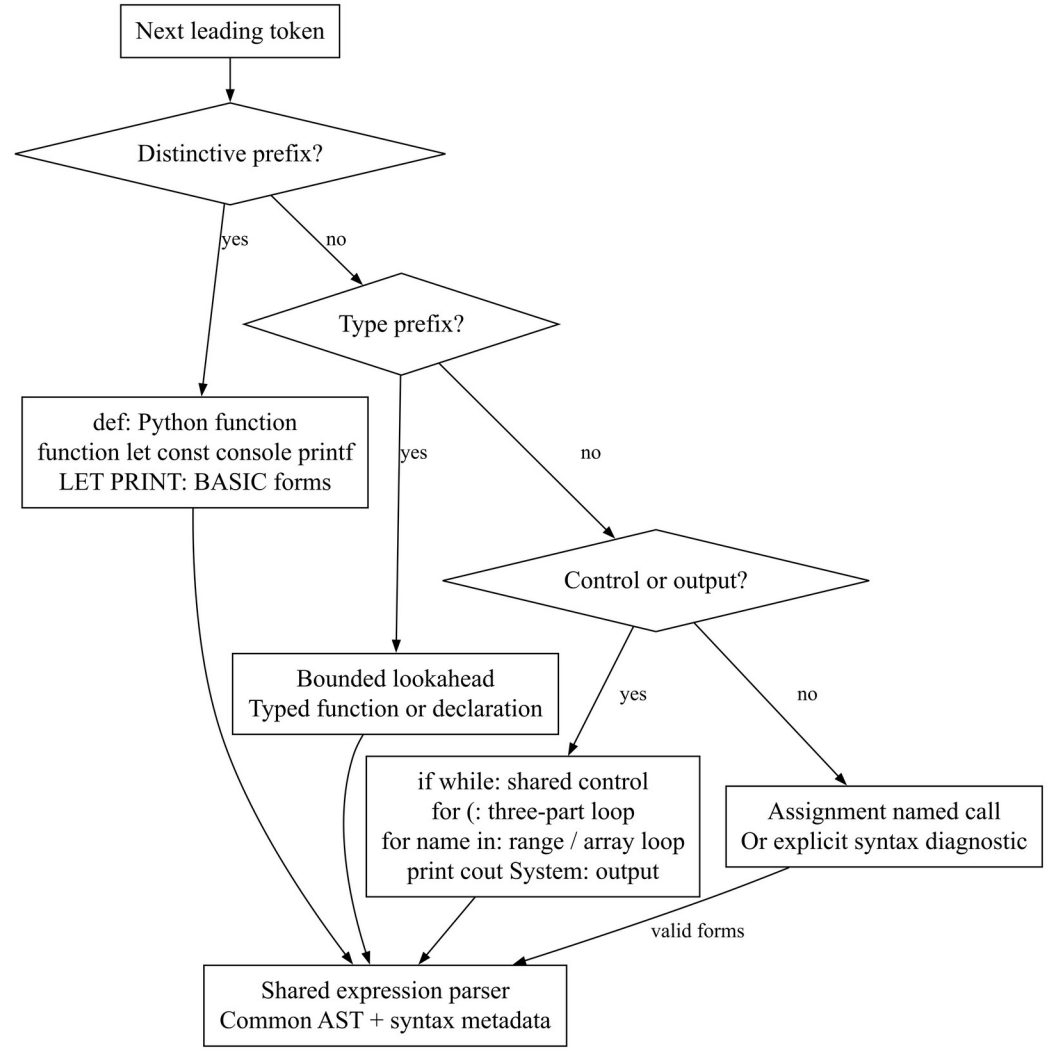


Figure 4 Grammar dispatch for supported statements

- A type prefix is checked to distinguish a variable declaration from a function definition. An opening parenthesis after for selects the three-part loop rule.
- The suite reader accepts braces or a colon followed by indentation. Its layout-context stack allows either block style to appear inside the other.
- Unsupported forms produce parser diagnostics. The dispatcher chooses a grammar rule rather than trying six independent language parsers.

System Design

The AST records expression structure, while the control-flow graph records possible paths through IR basic blocks. These views are used for different stages of analysis.

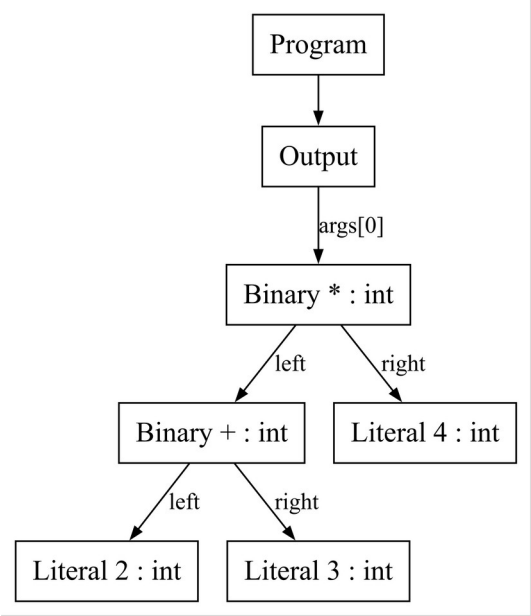


Figure 5 AST for print((2 + 3) * 4)

- The parentheses place addition below multiplication in the tree, producing 20. Without those parentheses, print(2 + 3 * 4) produces 14. Semantic analysis assigns int to these arithmetic nodes.

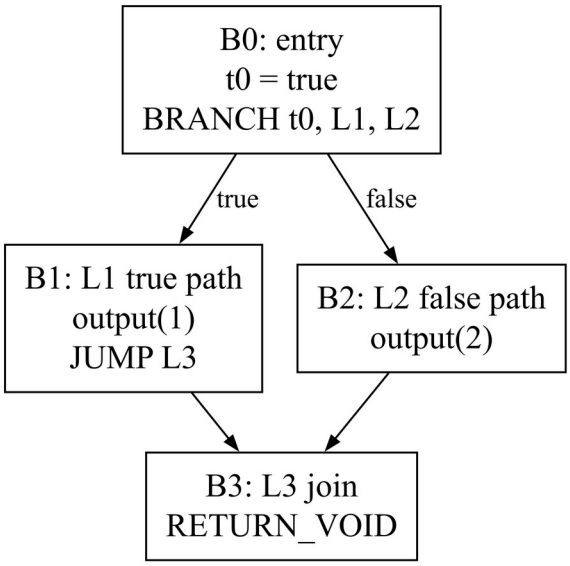


Figure 6 Original CFG for a constant conditional

- The original graph retains both branch targets. Once a known condition becomes a direct jump, the CFG is rebuilt and the unused branch can be removed. The illustrated optimized conditional prints 1.

System Design

All accepted function definitions use the same activation-record layout. In the mixed example, twice(3) calls add(3, 3) and returns 6. Globals are shared, while each call has its own local storage and operand stack.

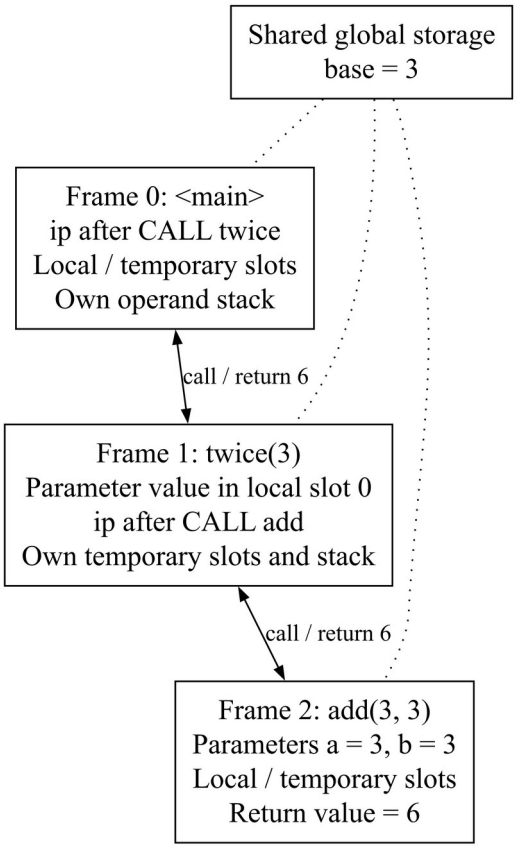


Figure 7 Frames for a call between mixed syntax functions

Representation	Main contents	Purpose
Token and Span	Kind, value and source range.	Lexing and source navigation.
Node	Kind, children, span and metadata.	AST structure and annotations.
Symbol and Scope	Type, slot, owner, mutability and parent.	Name resolution and storage.
IRFunction	Parameters, return type, instructions and temporaries.	Typed intermediate code.
Artifact and Frame	Executable records; per-call locals, ip and stack.	Validation and VM execution.

- Arguments are evaluated left to right. Each recursive call receives a fresh frame, and returning restores the caller. Arrays passed to functions retain shared references.

7. Technology Stack

Technology	Role	Recorded configuration
Python	Compiler, CLI, local server and VM.	Requires Python 3.11 or newer; verified with CPython 3.12.14.
Python standard library	HTTP, spawned workers, JSON, data structures and limits.	No third-party package needed for local execution.
HTML, CSS and JavaScript	Editor, stage tables, AST view and draft recovery.	Vanilla browser interface.
Custom bytecode and VM	Executable format and stack-based runtime.	New artifacts use bytecode version 3.
Pyodide and WebAssembly	Run the Python compiler in a static browser worker.	Runtime pinned to 314.0.7.
pytest	Python regression tests.	Verified with pytest 9.1.1.
Node.js test runner and jsdom	Frontend and DOM tests.	Recorded versions: Node v24.22.1 and jsdom 30.1.3.
GitHub Actions and Pages	Build and host the static browser package.	Project-relative assets and a packaged compiler.

- Python alone runs the local compiler and IDE. Node and jsdom are needed for frontend development tests; Pyodide is needed for the static browser version.
- Compiler modules are under src/polyglot/. Parsing is under parser/, and the interface and server are under web/. Analysis, optimization, artifact loading and VM execution have separate modules.
- The browser saves the current draft and layout preferences locally. Compiler structures remain in memory, and reusable programs are stored as JSON artifacts. [1]

8. Code and Implementation

- The lexer reads a Source object and produces tokens with their original text, value and span. It recognizes literals, identifiers, operators, comments and logical newlines. CRLF input is normalized before positions are calculated.
- Parser.statement chooses a rule from the leading tokens. The same expression parser is reused across syntax families, and the layout reader handles braces and indentation in context.

The token-emission helper keeps source positions attached to each token:

```
def emit(kind: str, start: int, end: int, value=None):
    tokens.append(Token(kind, text[start:end], value,
                        source.span(start, end)))
```

- Pratt parsing handles precedence through a table and a minimum binding power. Multiplication binds more tightly than addition. Logical spellings &&, || and ! become and, or and not. Unparenthesized chained comparisons are rejected.
- Normalization converts equivalent statements to common AST nodes. Semantic analysis collects function signatures before checking bodies, allowing forward calls and recursion.
- The analyzer checks names, scope, operator types, Boolean conditions, function arguments, returns and const rebinding. Each symbol records its type, storage slot, owning function and declaration position.

The shared compatibility rule permits int-to-float widening:

```
def compatible(self, node, actual, expected):
    if actual != expected and not (
        actual == "int" and expected == "float"
    ):
        self.error(node, "S004",
                   f"Cannot use {actual} where {expected} is required.")
```

- Type and initialization checks take place before optimization. A program cannot depend on the optimizer to hide a semantic error. These snippets are shortened from lexer.py and semantics.py. [1]

Code and Implementation

- Lowerer.lower converts the checked AST into typed three-address code. Operations refer to numbered temporaries, resolved storage slots and function IDs. LABEL, JUMP and BRANCH represent control flow.
- Operands and call arguments keep source order. Logical and/or lower to branches so the second operand is evaluated only when needed. Range arguments and array iterables are evaluated once.
- Analysis splits each function into basic blocks, resolves successor edges and marks reachability from the entry block. Temporary liveness is then solved backwards until the sets stop changing. [4]

```
OUT[B] = union of IN[S] for each successor S
IN[B]  = USE[B] union (OUT[B] - DEF[B])
```

- A reverse scan records the next use of each temporary and distinguishes successive definitions. These facts help decide whether a constant definition is still needed.
- The optimizer copies eligible IR, folds exact primitive constants, simplifies known branches and removes unreachable blocks. It deletes only safe, unused CONST definitions.
- Reachable calls, loads, stores, output, array checks and operations that may fail are retained. For example, division by zero keeps its runtime error and source location.
- Analysis is bounded to 12 functions, 1,500 instructions per function, 3,000 instructions overall and 120 blocks per function. Oversized functions keep their original IR and report an optimization skip.

```
for each eligible function:
    build CFG and track local constants
    fold safe results and simplify known branches
    rebuild CFG and remove unreachable blocks
    solve liveness and next-use information
    remove safe unused CONST definitions
```

Code and Implementation

- Code generation translates typed IR into stack instructions. A binary operation loads its input temporaries, performs BINARY and stores the result. Temporary slots follow the declared local slots.
- Jump targets are symbolic during lowering. Code generation emits placeholders and records their positions, then patches them with absolute instruction indexes after all labels are known.

```
for position, target in jumps:
    code[position]["args"][0] = labels[target]
```

- A .pbc file is UTF-8 JSON containing the format, version, entry function, constants, globals and function records. Instructions include source metadata. New files use version 3, with older supported scalar and array versions still accepted.
- The loader rejects duplicate keys and invalid shapes. Validation checks instructions, operands, storage indexes, call arities and branch targets. Stack-flow analysis rejects underflow, incompatible merge heights and invalid returns. Artifacts are limited to five megabytes.
- The VM validates the artifact and starts an entry frame. Each instruction advances an instruction pointer and increments the step counter. Calls create a frame, and returns restore the caller and pass back a result when required.
- Arrays store a fixed element type and a list of values. Indexed reads and writes check the array, integer index and bounds. Assignments and calls share array references.

Default limit	Value
Executed instructions	1,000,000
Recursion depth	64
Output	65,536 bytes
VM execution time	5 seconds

- Runtime failures keep partial output and attach the current source location and call stack. PolyGlot source is executed as custom bytecode, without Python eval or exec. [1]

Code and Implementation

The worked input print((2 + 3) * 4) produces 20. It has 12 tokens including NEWLINE and EOF, seven AST nodes and no user-defined bindings. The compiler first generates the following typed IR:

```
t0 = CONST 2 : int
t1 = CONST 3 : int
t2 = BINARY + t0, t1 : int
t3 = CONST 4 : int
t4 = BINARY * t2, t3 : int
OUTPUT t4, separator="", ending="\n"
RETURN_VOID
```

Constant folding computes 5 and then 20. Four unused constant definitions are removed, leaving:

```
t4 = CONST 20 : int
OUTPUT t4, separator="", ending="\n"
RETURN_VOID
```

Index	Unoptimized bytecode	Optimized bytecode
0	CONST 0 (2)	CONST 0 (20)
1	STORE_LOCAL 0	STORE_LOCAL 4
2	CONST 1 (3)	LOAD_LOCAL 4
3	STORE_LOCAL 1	OUTPUT 1, "", "\n"
4	LOAD_LOCAL 0	RETURN_VOID
5	LOAD_LOCAL 1	
6	BINARY "+"	
7	STORE_LOCAL 2	
8	CONST 2 (4)	
9	STORE_LOCAL 3	
10	LOAD_LOCAL 2	
11	LOAD_LOCAL 3	
12	BINARY "*"	
13	STORE_LOCAL 4	
14	LOAD_LOCAL 4	
15	OUTPUT 1, "", "\n"	
16	RETURN_VOID	

- Both artifacts print 20. Executed instructions fall from 17 to 5 for this straight-line example. Slot 4 remains because temporary numbering is not compacted; this result does not measure elapsed time.

9. Testing and Validation

Check	Recorded result
Python regression suite	298 passed; 0 failed. Includes CLI, artifact, HTTP and packaging tests.
Frontend regression suite	57 passed; 0 failed with PYTHONUTF8=1.
Static package build	Completed successfully and generated build/pages.
Optimization comparison	Output 20 in both modes; 17 versus 5 instructions.
Saved artifact execution	Output 20 in 5 steps without source compilation.
Hosted browser example	Output 20 in 5 steps after runtime initialization.

Case	Expected and observed result	Status
mixed_demo.poly	0, 1, 3, 6, 12 and done on separate lines.	Pass
Mixed nested blocks	mixed_nesting.poly prints 22.	Pass
Operator precedence	print(2 + 3 * 4) prints 14.	Pass
Recursive calls	recursion.poly prints 720 in 168 steps.	Pass
Arrays and functions	15; 7; [1, 7, 3, 4] 4 in 307 steps.	Pass
Short circuit	false and true results without division failure.	Pass
Array iteration	array_iteration.poly prints 1 through 5.	Pass

Testing and Validation

Case	Expected and observed behavior	Status
Type mismatch	count = 3 followed by count = "three" gives S004 at 2:1. No artifact is produced.	Pass
Out-of-scope name	Reading a local outside its block gives S003 at 3:7.	Pass
Invalid expression	print(2 +) gives P010 at 1:11.	Pass
Array bounds	Reading index 2 from [1, 2] gives R012 at 2:7.	Pass
Instruction limit	An infinite loop with a 30-step limit gives R001 and retains partial output.	Pass
Artifact reuse	A saved artifact runs after its source file is removed.	Pass

Integration check	Recorded outcome
HTTP operations	Check, Run, Build, Inspect and Exec returned shared compiler results.
Worker deadline	An overdue job was stopped, and a later request succeeded.
Draft recovery	Source and filename were restored after refresh, including an empty draft.
Stale responses	Older diagnostics did not link into edited source.
AST controls	Expand, focus, orientation and zoom retained node relationships.
Artifact upload	Execution did not claim unrelated source stages.
Pages package and adapter	Relative assets, compiler packaging, readiness, errors and cancellation passed their tests.

- Manual local-browser checks covered tokens, symbols, the connected AST, original IR, bytecode, VM output and a semantic error that blocked later stages.
- The hosted run covered one arithmetic example. The repository also records an earlier set of 26 WebAssembly cases; that set was not repeated in the report verification.

10. Results and Screenshots

The mixed example defines add with C-style syntax, defines twice with Python-style syntax, assigns base with BASIC LET and prints through console.log. The call twice(3) returns 6 in 32 VM instructions.



Figure 8 Mixed syntax program and output

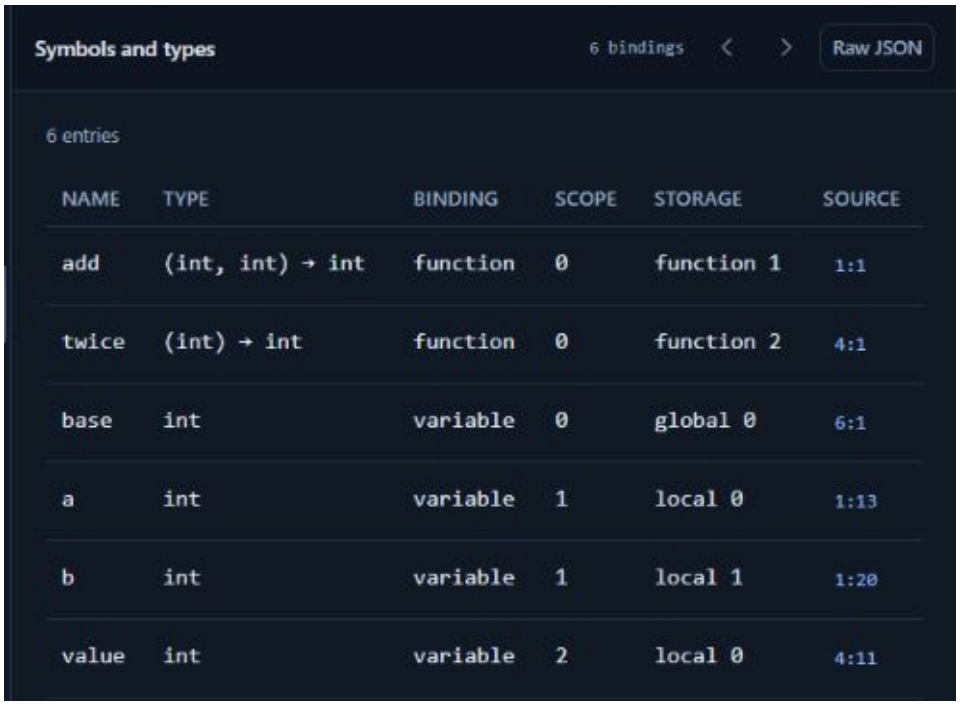


Figure 9 Shared symbols and storage slots

- The symbol view records add(int, int) and twice(int), both returning int. The functions and base belong to root scope 0; each parameter has a local slot in its function scope.

Results and Screenshots

For print((2 + 3) * 4), the lexer returns positioned tokens and semantic analysis adds int annotations to the AST. These screenshots show the actual inspector data.

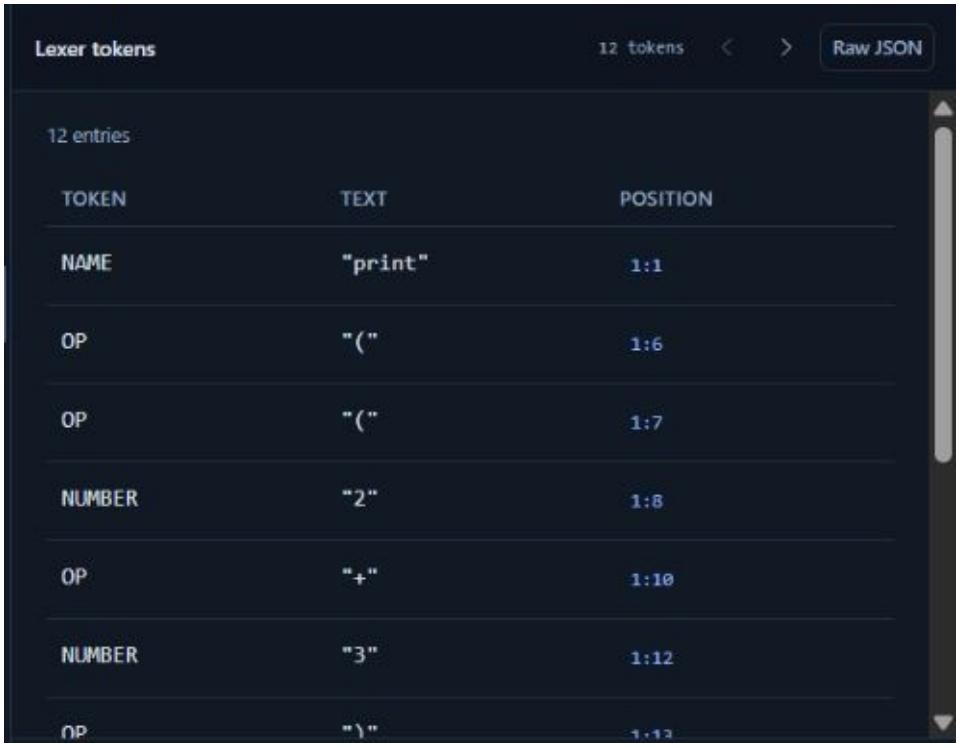


Figure 10 Tokens with text and source positions

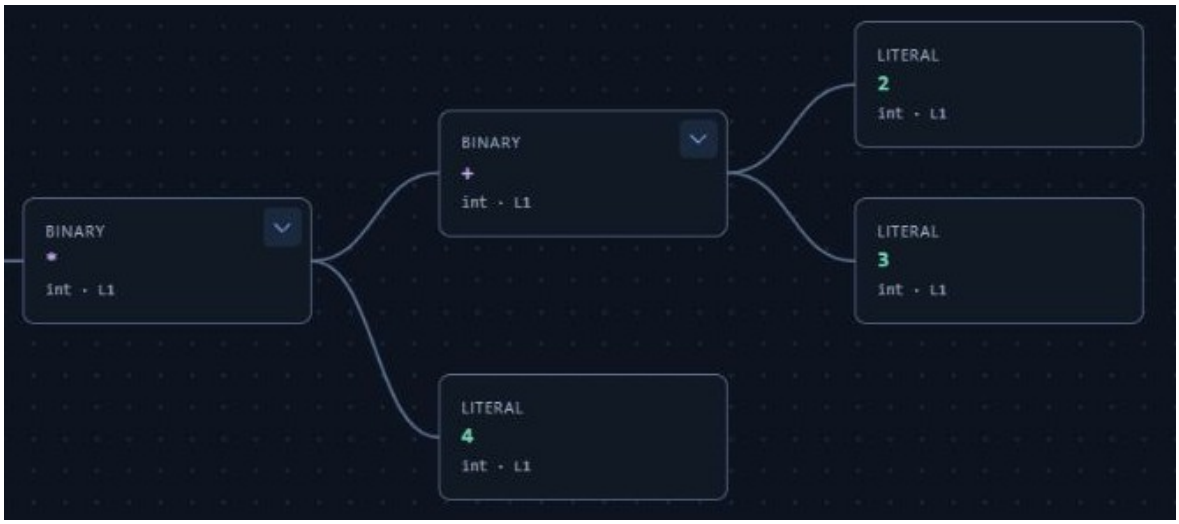


Figure 11 Connected arithmetic subtree in the AST viewer

- The full token response contains 12 entries, including NEWLINE and EOF. NAME, OP and NUMBER entries preserve the source spelling and one-based positions.
- The multiplication node takes the addition node as its left operand and literal 4 as its right operand. The addition node connects to literals 2 and 3.

Results and Screenshots

The Typed IR panel keeps the original seven operations visible. Bytecode generation uses the optimized IR, producing five instructions for the same expression.

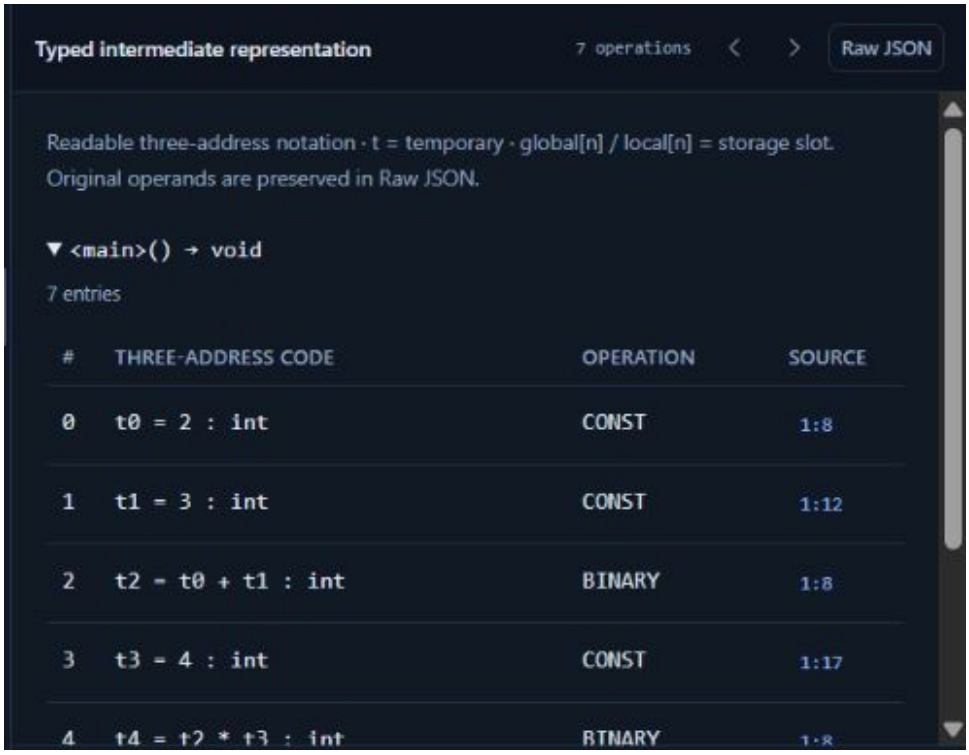


Figure 12 Original typed IR before optimization

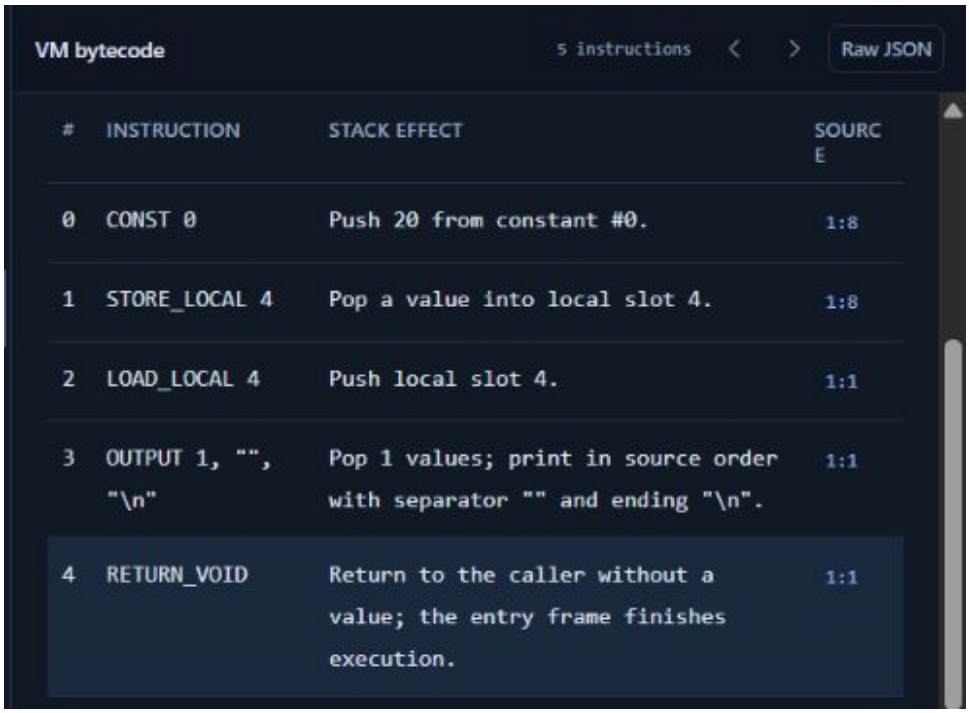


Figure 13 Generated bytecode after optimization

- The generated sequence loads constant 20, stores and reloads slot 4, outputs the value and returns. Keeping both stages visible makes the effect of constant folding easy to compare.

Results and Screenshots

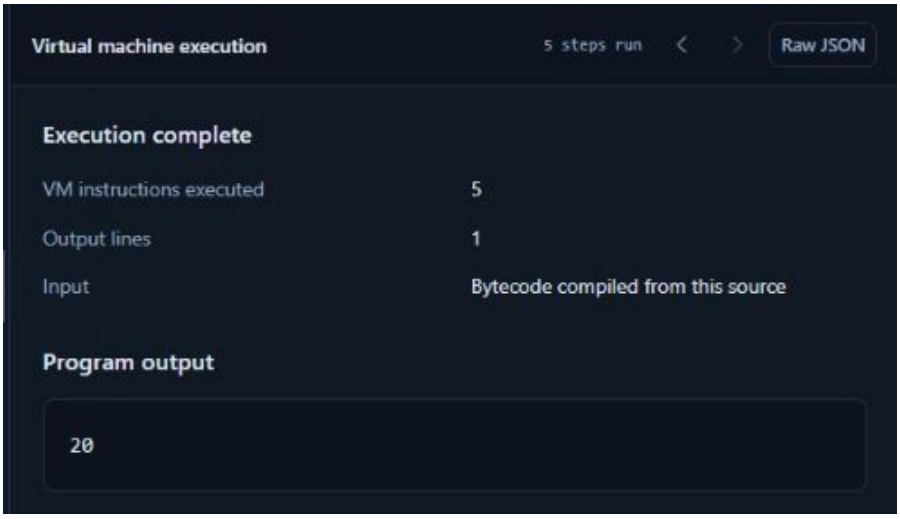


Figure 14 Output 20 in five VM instructions

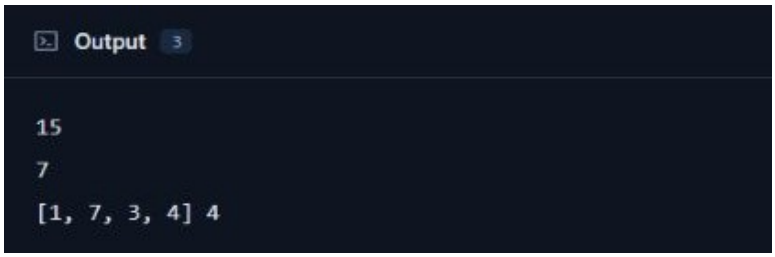


Figure 15 Array and function example output

- The array example prints 15, then 7, then [1, 7, 3, 4] 4. The indexed update is visible to the shared sum function, and the run uses 307 instructions.



Figure 16 Source with an incompatible assignment

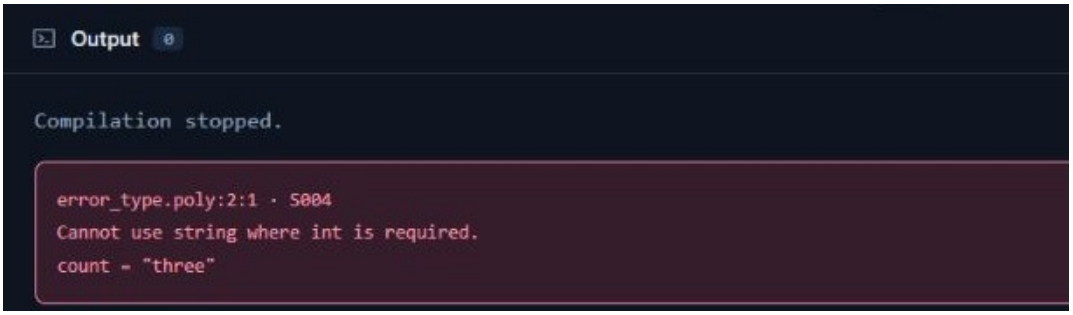


Figure 17 S004 diagnostic from semantic analysis

- Assigning "three" to the int binding count reports S004 at line 2, column 1. Tokens and AST remain available; later compilation stages and execution are blocked.

11. Repository and Setup

Repository: https://github.com/hst257/PolyGlot
Live site: https://hst257.github.io/PolyGlot/

The report uses project version 0.3.0 at commit 9aee210c617e394958178c91df790ad72ec50722. Public hosting was recorded as available on 26 October 2026. [1]

Path	Contents
src/polyglot/	Compiler, parser, analysis, artifact loader, VM, CLI and web interface.
examples/	Accepted programs and intentional error cases.
tests/ and tests/web/	Python and frontend regression tests.
docs/	Language rules, grammar, architecture, bytecode and verification.
tools/ and .github/workflows/	Static-site packaging and deployment.

- Install Python 3.11 or newer and run these commands from the repository folder. On Windows, use python in place of python3. Keep the server terminal open while using the IDE.

```
python3 run_polyglot.py serve
# Open http://127.0.0.1:8765
python3 run_polyglot.py check examples/mixed_demo.poly
python3 run_polyglot.py run examples/mixed_demo.poly
python3 run_polyglot.py build examples/mixed_demo.poly -o build/demo.pbc
python3 run_polyglot.py exec build/demo.pbc
```

- Development tests use pytest and the frontend dependencies. On Windows, enable PYTHONUTF8=1 for the frontend test subprocess.

```
python3 -m pip install pytest
python3 -m pytest -q
npm ci
npm test
```

- For the static browser version, download the pinned runtime while building the package, then serve build/pages locally. The hosted version uses the same compiler modules.

```
python3 tools/build_pages.py --download-runtime
python3 -m http.server --directory build/pages
```

12. Challenges and Solutions

- Some statements fit more than one syntax family. Instead of assigning a false single origin, recognition metadata stores candidate families. They all normalize to the same semantic node.
- Indentation inside braces can be formatting or the start of an indented suite. A parser-driven layout stack tracks the active block style and checks indentation only where it is meaningful. The mixed-nesting example prints 22.
- Different declarations and function spellings need consistent type rules. Shared symbols, parent-linked scopes and normalized type names provide one checking policy. Collecting signatures first supports forward calls and recursion.
- Short-circuit expressions must not evaluate a skipped operand. The lowerer emits branches and MOVE operations for and/or, while arguments and array inputs retain their specified evaluation order.
- An unused computation can still call a function or raise an error. The optimizer removes only safe constant definitions and unreachable blocks. Reachable operations with effects or possible failures are retained.
- Errors after code generation still need a useful source location. Spans flow from tokens through AST, IR and bytecode. Artifacts retain source excerpts so runtime diagnostics work without reopening the source file.
- Uploaded artifacts may contain invalid instructions or storage references. Structural checks and stack-flow validation reject malformed programs, while runtime limits bound instructions, recursion, output and execution time.
- A static site cannot use a local spawned Python process. A Pyodide Web Worker runs the shared compiler API in the browser, and the JavaScript adapter keeps the same request and result contract.

13. Conclusion and Future Scope

PolyGlot compiles its documented mixed-syntax subset through one shared implementation. Accepted forms can use common variables, typed functions, arrays and runtime rules. The inspector connects source text with the tokens, AST, typed IR, bytecode and output produced by the compiler.

The recorded tests support the implemented subset: 298 Python tests and 57 frontend tests passed. Mixed nesting prints 22, recursion prints 720, and the arithmetic example prints 20 with either optimization setting. Saving and reusing .pbc artifacts separates compilation from execution. The project remains a teaching compiler with explicit language and runtime limits. [1]

The following extensions would improve the current implementation:

- Add error recovery so one compilation can report several useful diagnostics. Recovery should preserve source locations and avoid misleading errors caused by an earlier failure.
- Extend syntax only after defining its shared semantics and adding positive and negative tests. Labels, richer collections and member operations would need corresponding language and analysis changes.
- Investigate inter-block constant propagation, copy propagation and frame-slot compaction while preserving side effects and runtime failures.
- Add stepping, breakpoints, operand-stack views and frame inspection. The current VM view reports output and an instruction count.
- Use a larger benchmark collection and repeat browser-runtime tests. Timing comparisons should record the environment, workloads and repeated trials; the present 17-to-5 result measures one program’s instruction count.

14. References

[1] PolyGlot contributors. PolyGlot source, documentation, examples and tests. Version 0.3.0. Commit 9aee210c617e394958178c91df790ad72ec50722. https://github.com/hst257/PolyGlot

[2] LLVM Project. My First Language Frontend with LLVM Tutorial. https://llvm.org/docs/tutorial/MyFirstLanguageFrontend/index.html

[3] Pratt, V. R. Top down operator precedence. POPL, 1973, pp. 41–51. DOI: 10.1145/512927.512931. https://doi.org/10.1145/512927.512931

[4] Cornell University. CS 4120 compiler notes, Spring 2023. Dataflow analysis and live variables. https://courses.cs.cornell.edu/courses/cs4120/2023sp/notes/

[5] Compiler Explorer contributors. Official source and project documentation. https://github.com/compiler-explorer/compiler-explorer

[6] AST Explorer contributors. Official project README. https://github.com/fkling/astexplorer

[7] Nystrom, R. Crafting Interpreters. https://craftinginterpreters.com/

[8] Pyodide contributors. Using Pyodide in a web worker. https://pyodide.org/en/stable/usage/webworker.html

[9] Python Software Foundation. http.server. Python 3.12 documentation. https://docs.python.org/3.12/library/http.server.html

[10] Python Software Foundation. multiprocessing. Python 3.12 documentation. https://docs.python.org/3.12/library/multiprocessing.html

[11] pytest contributors. Official documentation. https://docs.pytest.org/en/stable/

[12] Node.js contributors. Test runner documentation. https://nodejs.org/api/test.html

[13] jsdom contributors. Official project README. https://github.com/jsdom/jsdom

Appendix A Selected Source Code

These excerpts show the main implementation decisions. They are selected from src/polyglot/, with surrounding bookkeeping omitted and long statements wrapped for readability. Existing lexer, type checking and backpatching excerpts appear in Section 8.

A1 Shared compilation pipeline

Source: compiler.py, compile_source. The same frontend and backend serve the CLI and browser workers.

```
  tokens = lex(source)
  parser = Parser(source, tokens)
  tree = parser.parse()
  analyzer = Analyzer(source)
  symbols = analyzer.analyze(tree)
  functions = Lowerer(source, analyzer).lower(tree)
  if optimize:
      functions, stages["optimization"] = optimize_functions(functions)
  artifact = generate(functions, symbols)
  dumps(artifact)
```

The parser produces one AST. The analyzer resolves names and types before lowering. Optimization changes the IR passed to generate; dumps validates and serializes the artifact. The surrounding error handler returns no artifact if a compiler diagnostic is raised.

A2 Pratt expression parsing

Source: parser/shared.py, expression. This is the binary-node construction inside the precedence loop:

```
  op = p.take()
  # Chained-comparison validation occurs here in the full function.
  right = expression(p, PRECEDENCE[op.text] + 1)
  left = Node(
      "Binary", p.join(left.span, right.span),
      {"op": NORMAL.get(op.text, op.text),
       "left": left, "right": right}
  )
```

The loop accepts operators whose binding power meets minimum. Recursing with precedence + 1 makes equal-precedence binary operators associate to the left, while higher-precedence operators bind inside the right operand. Parentheses are handled before this loop; source spans remain attached to the node.

Appendix A Intermediate Code and Optimization

A3 Typed three address code

Source: ir.py, Lowerer.binary. One expression creates a temporary and a typed instruction:

```
def binary(self, operator, left, right, node, type_name):
    dest = self.temp()
    self.emit("BINARY", node, dest=dest, operator=operator,
              left=left, right=right, type=type_name)
    return dest
```

left and right refer to earlier temporaries. dest identifies the new result, type_name records its checked type, and node supplies its source location. This representation separates the expression from any particular stack instruction sequence.

A4 Safe constant folding

Source: optimizer.py, optimize_functions. The actual folding decision is:

```
result = constant_result(instruction, constants)
if result is not UNKNOWN and safe_constant(
    result, instruction.get("type", "")
):
    instruction = {
        "op": "CONST", "dest": instruction["dest"],
        "value": result, "type": instruction["type"],
        "source": instruction["source"]
    }
    report["constant_folds"] += 1
```

Only known, valid primitive results become CONST instructions. A failed calculation, such as division by zero, remains a runtime operation. Constant facts are local to a basic block, so mutable bindings and control flow joins do not inherit unsupported assumptions.

A5 Removing unreachable blocks

```
cfg = analyze_function(asdict(fn))
reachable = []
for block in cfg["blocks"]:
    instructions = fn.instructions[block["start"]:block["end"]]
    if block["reachable"]:
        reachable.extend(instructions)
    else:
        report["unreachable_removed"] += len(instructions)
fn.instructions = reachable
```

After branch simplification, rebuilding the CFG identifies blocks with no path from entry. They can be removed without executing them. Liveness and next-use analysis subsequently remove only safe unused constant definitions, as described in Section 8.

Appendix A Bytecode and VM Execution

A6 Generating stack instructions

Source: codegen.py, generate. A BINARY IR instruction becomes:

```
elif op == "BINARY":
    load(inst["left"], loc)
    load(inst["right"], loc)
    emit("BINARY", [inst["operator"]], loc)
    store(inst["dest"], loc)
```

The helpers emit LOAD_LOCAL and STORE_LOCAL using declared local count + temporary number. Loading left before right preserves operand order. The bytecode operation consumes the two operand values and produces one result; the store saves it for later instructions.

A7 Fetch decode and execute

Source: vm.py, run. Each cycle selects the active activation record and reads its next instruction:

```
while frames:
    frame = frames[-1]
    instruction = frame.function["code"][frame.ip]
    frame.ip += 1
    steps += 1
    op, args = instruction["op"], instruction["args"]
```

The VM checks instruction and time limits before dispatch. A constant pushes its value onto the operand stack; the binary handler pops right and left, applies the validated operator and pushes the checked result. STORE_LOCAL pops that value into frame storage.

A8 Function activation records

The CALL handler checks recursion depth, pops and orders arguments, resolves the callee and checks parameter types. It then allocates a new frame:

```
locals_ = [UNINITIALIZED] * callee["local_count"]
locals_[:len(values)] = values
frames.append(Frame(
    callee, locals_, callsite=instruction["source"]
))
```

Each call has independent local slots, an instruction pointer and an operand stack. RETURN_VALUE removes the callee frame and pushes the result onto the caller’s stack. This is how recursion and calls across syntax families use one execution model.